Actividad Evaluativa #1: Centro de Monitoreo de Accesos

Programación Orientada a Objetos — Grado en Ciberseguridad

ENTI – Universitat de Barcelona

23 de octubre de 2026

1. Introducción y Objetivos

El presente informe documenta el desarrollo del sistema de análisis y monitoreo de autenticación para dos servidores independientes. La solución implementa el paradigma de Programación Orientada a Objetos (POO) en Java, encapsulando en la clase `Servidor` el estado y los métodos de validación, cálculo de métricas y evaluación de riesgo bajo directrices de seguridad.

2. Diseño Orientado a Objetos

El sistema consta de dos clases estructuradas en el archivo `Main.java`:

- **Clase Servidor:** Modela la entidad del servidor con cinco atributos de instancia (`nombre`, `origen`, `critico`, `intentosTotales`, `intentosFallidos`). Expone métodos para:
  - `datosValidos()`: Validación lógica y saneamiento de entradas (`trim()`, `toLowerCase()`).
  - `evaluarEstado()`: Asignación jerárquica de niveles de severidad (`BLOQUEADO`, `ALERTA`, `NORMAL`, `DATOS INVALIDOS`).
  - `porcentajeFallos()`: Cálculo fraccionario en coma flotante (`double`).
  - `mostrarResumen()`: Presentación formateada de registros de seguridad.
- **Clase Main:** Orquesta la entrada por consola mediante `Scanner`, instancia dos objetos independientes de tipo `Servidor` y determina la prioridad operativa.

3. Respuestas a las Preguntas Conceptuales

1. **¿Por qué dos objetos de la misma clase pueden almacenar valores diferentes en sus atributos?**
   Porque una clase define la estructura o molde tipificado, mientras que la invocación del operador `new` instancia un espacio de memoria independiente (en el *heap*) para cada objeto. Cada entidad posee su propio bloque de variables de instancia, permitiendo que la mutación de estado en un objeto no afecte al otro.

2. **¿Qué diferencia existe entre `Servidor b = a;` y `Servidor b = new Servidor();`?**
   La sentencia `Servidor b = new Servidor();` construye físicamente un nuevo objeto en memoria y asigna su puntero/referencia a la variable `b`. En contraste, `Servidor b = a;` únicamente copia la dirección de memoria referenciada por `a` en `b`; ambos identificadores apuntan a la misma instancia subyacente, compartiendo cualquier modificación posterior.

3. **¿Por qué es importante comprobar `datosValidos()` antes de calcular el porcentaje?**
   Por consistencia numérica y prevención de anomalías en tiempo de ejecución:

   - Si `intentosTotales` fuese 0, la operación aritmética provocaría una indeterminación matemática (`NaN` o `Infinity` en tipos `double`).
   - Si la muestra no satisface los axiomas del modelo (por ejemplo, registros con fallos superiores al total), el porcentaje resultante generaría falsas métricas en los informes de auditoría.

4. Casos de Prueba y Resultados

A continuación se detalla la matriz de pruebas ejecutada para verificar el cumplimiento de las reglas de clasificación:

Servidor	**Origen**	**Crítico**	**Totales**	**Fallidos**	**Estado Obtenido**
VPN	externo	si	10	3	BLOQUEADO
WEB	interno	no	8	4	ALERTA
CORREO	externo	no	8	2	ALERTA
VPN	interno	si	12	2	NORMAL
WEB	externo	no	5	5	BLOQUEADO
WEB	interno	no	4	5	DATOS INVALIDOS
WEB	interno	no	0	0	DATOS INVALIDOS
CORREO	local	no	10	2	DATOS INVALIDOS
WEB	externo	quizás	10	2	DATOS INVALIDOS
VPN	.EXTERNO ”	”SI ”	10	3	BLOQUEADO (Trim OK)

Cuadro 1: Matriz de validación y casos límite.